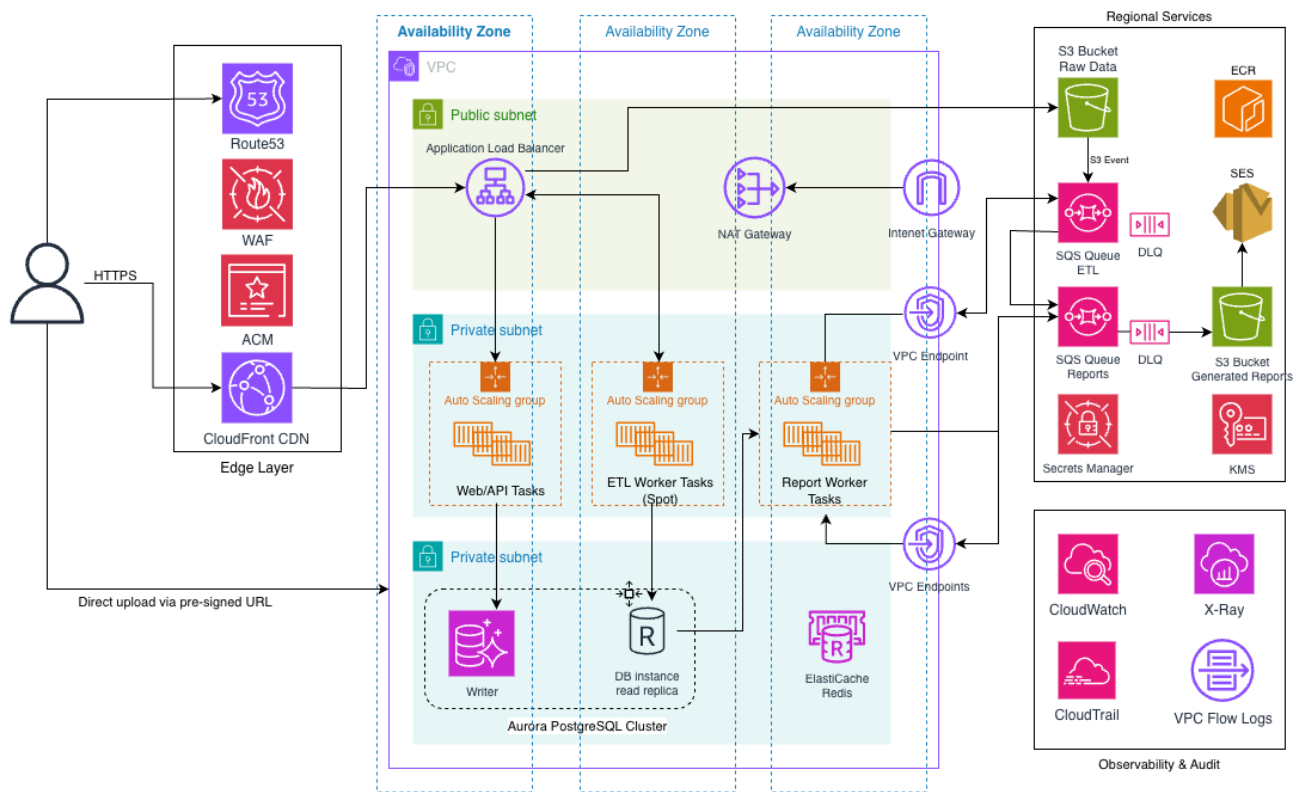


ARCHITETTURA

Di seguito il diagramma di alto livello dell'architettura implementata:



SEZIONE 1: DESCRIZIONE DEL FLUSSO DEI DATI

1.1 Panoramica dei Flussi

L'architettura gestisce **quattro flussi dati principali**, progettati per essere completamente disaccoppiati tra loro, in modo che nessuna operazione di background possa degradare l'esperienza utente sulla dashboard.

1.2 FLUSSO 1 — Accesso Utente alla Dashboard (Lettura Real-Time)

Utente → Route 53 → CloudFront (+ WAF + TLS) → ALB → Fargate Web/API → ElasticCache Redis (cache check) → (cache miss) Aurora Reader

Percorso dettagliato:

1. **Risoluzione DNS:** L'utente digita dashboard.acme.com. Route 53, configurato con un **Alias Record**, risolve il dominio verso la distribuzione CloudFront. Si utilizza un Alias Record anziché

un CNAME perché è nativo AWS, gratuito in termini di query DNS, e risolve direttamente all'endpoint CloudFront senza hop aggiuntivi.

2. **Edge Layer — CloudFront + WAF:** La richiesta raggiunge il **Point of Presence (PoP) CloudFront** geograficamente più vicino all'utente. Qui avvengono simultaneamente tre operazioni critiche:

- **Terminazione TLS** con certificato ACM: la connessione HTTPS viene terminata all'edge, riducendo la latenza percepita dall'utente perché il TLS handshake avviene nel nodo più vicino anziché nel data center di origine.
- **AWS WAF** ispeziona la richiesta applicando regole contro SQL Injection, XSS, rate limiting e blocco di IP malevoli. Le richieste che violano le regole vengono scartate prima ancora di raggiungere l'infrastruttura di backend.
- **Cache statica:** per gli asset statici della dashboard (JavaScript, CSS, immagini, font), CloudFront serve direttamente dalla cache edge senza contattare l'origine, riducendo drasticamente il carico sull'ALB e sui container.

3. **Application Load Balancer:** Le richieste dinamiche (API calls) che superano CloudFront raggiungono l'ALB nelle **subnet pubbliche**. L'ALB esegue:

- **Health check** continui sulle Fargate Tasks (endpoint /health) ogni 30 secondi, rimuovendo automaticamente dal target group le istanze non sane.
- **Distribuzione del traffico** verso le Fargate Tasks Web/API distribuite su entrambe le Availability Zones.
- Il listener HTTPS sull'ALB fornisce un secondo livello di terminazione TLS per il traffico interno proveniente da CloudFront.

4. **Fargate Web/API Tasks:** I container ricevono la richiesta e implementano un **pattern Cache-Aside** (chiamato anche Lazy Loading) con ElastiCache Redis:

- **Cache Hit:** se il dato richiesto (ad esempio un aggregato della dashboard, il risultato di una query frequente, o lo stato di un job) è presente in Redis, viene restituito immediatamente con latenza sub-millisecondo. Questo è il caso più comune per le dashboard, dove molti utenti visualizzano gli stessi dati aggregati.
- **Cache Miss:** il container interroga l'**Aurora Reader** endpoint, che instrada automaticamente la query verso una delle repliche di lettura. Il risultato viene simultaneamente restituito all'utente e scritto in Redis con un **TTL appropriato** (tipicamente 30-60 secondi per dati della dashboard, più lungo per dati storici che cambiano raramente).

5. **Gestione delle Sessioni:** Le sessioni utente sono memorizzate in Redis anziché nei container, rendendo il layer Web/API completamente **stateless**. Questo significa che qualsiasi Fargate

Task può servire qualsiasi richiesta di qualsiasi utente, abilitando l'auto-scaling orizzontale senza perdita di sessione e permettendo rolling deployments senza disconnettere gli utenti.

Perché questo flusso evita colli di bottiglia:

- CloudFront assorbe il traffico statico e gli spike di richieste identiche.
- Redis elimina la maggior parte delle query ripetitive verso Aurora.
- Le Aurora Reader replicas si auto-scalano da 1 a 5 in base al carico.
- Le Fargate Web/API scalano indipendentemente dai worker ETL e Report.

1.3 FLUSSO 2 — Caricamento File Pesanti (>2 GB)

Questo è il flusso che richiede la massima attenzione architetturale, perché il caricamento di file Excel/CSV superiori a 2 GB rappresenta la principale fonte potenziale di colli di bottiglia. La soluzione adottata è il **pattern Pre-signed URL con Multipart Upload diretto a S3**, che bypassa completamente l'infrastruttura applicativa durante il trasferimento dati.

Utente → Web/API (richiesta upload)

Web/API → S3 (genera Pre-signed URL)

Web/API → Utente (restituisce Pre-signed URL)

Utente → S3 Raw Bucket (upload diretto, multipart)

Step 1 — Richiesta di Upload

L'utente, attraverso l'interfaccia della dashboard, seleziona il file da caricare (Excel o CSV, potenzialmente >2 GB). Il frontend invia una richiesta API leggera (pochi byte) al servizio Web/API.

Questa richiesta transita normalmente attraverso CloudFront → ALB → Fargate, ed è una chiamata leggera che non crea alcun carico significativo.

Step 2 — Generazione del Pre-signed URL

La Fargate Task Web/API, senza mai toccare il file, esegue le seguenti operazioni:

- **Validazione:** verifica che il tipo di file sia consentito (Excel/CSV), che la dimensione dichiarata sia entro i limiti accettabili, e che l'utente abbia i permessi per caricare.
- **Generazione della chiave S3:** crea un path univoco nel bucket Raw, tipicamente strutturato per organizzare i dati e prevenire collisioni.
- **Generazione del Pre-signed URL:** attraverso l'SDK AWS, genera un URL pre-firmato per un'operazione PutObject (o, per file >100 MB, inizializza un **Multipart Upload** generando pre-signed URLs per ciascuna parte). Il pre-signed URL ha una **scadenza limitata** e include

nella firma le condizioni di upload (content-type, dimensione massima, chiave di destinazione), impedendo qualsiasi uso improprio.

- **Registrazione del job:** inserisce un record in Aurora (via Writer) o in Redis per tracciare lo stato dell'upload ("initiated"), permettendo al frontend di mostrare lo stato.

Step 3 — Upload Diretto a S3

Il browser/client dell'utente carica il file **direttamente su S3**, senza che i dati transitino attraverso CloudFront, ALB, o i container Fargate. Questo è fondamentale per diverse ragioni:

- **Nessun consumo di risorse applicative:** un file da 2 GB che transitasse attraverso i container Fargate occuperebbe CPU, memoria e bandwidth per minuti, bloccando potenzialmente la capacità del container di servire altri utenti. Con il pre-signed URL, i container non sono coinvolti nel trasferimento e restano liberi di servire le richieste della dashboard.
- **Nessun timeout:** le connessioni HTTP attraverso ALB hanno timeout configurati (tipicamente 60-120 secondi). Un upload da 2 GB su una connessione lenta potrebbe richiedere molto più tempo. L'upload diretto a S3 non ha questi vincoli.
- **Multipart Upload nativo S3:** per file >100 MB, il client divide il file in parti (tipicamente 100 MB ciascuna, quindi ~20 parti per un file da 2 GB) e le carica in parallelo. Questo offre:
 - **Parallelismo:** più parti caricate simultaneamente sfruttano meglio la banda disponibile.
 - **Resilienza:** se una parte fallisce, solo quella parte viene ricaricata, non l'intero file.
- **Throughput S3 praticamente illimitato:** S3 è progettato per gestire migliaia di richieste PUT al secondo con throughput aggregato. Un singolo upload, anche da 2 GB, non impatta minimamente le performance del bucket.
- **Encryption at rest automatica:** il bucket è configurato con **SSE-KMS** (Server-Side Encryption con AWS KMS), quindi ogni oggetto viene crittografato automaticamente al momento della scrittura utilizzando una chiave gestita in KMS. L'utente non deve fare nulla; la crittografia è trasparente e imposta come default bucket policy.

Step 4 — Notifica di Completamento e Innesco della Pipeline ETL

Al completamento dell'upload, S3 genera automaticamente un **S3 Event Notification**. Questo evento viene recapitato alla **coda SQS ETL Ingestion Jobs**.

La scelta di S3 Event → SQS (anziché S3 Event → Lambda → SQS o altre combinazioni) è deliberata:

- **Affidabilità:** SQS garantisce la consegna del messaggio con retention fino a 14 giorni. Se tutti i worker ETL sono occupati o non ci sono worker attivi (scale-to-zero), il messaggio resta in coda senza essere perso.
- **Disaccoppiamento temporale:** il produttore (S3) e il consumatore (ETL worker) non devono essere attivi contemporaneamente.

- **Backpressure naturale:** se arrivano 100 file contemporaneamente, i 100 messaggi si accodano e vengono elaborati al ritmo sostenibile dai worker, senza sovraccaricare nessun componente.

Riepilogo dell'impatto sui componenti durante un upload da 2 GB:

Componente	Carico durante upload	Durata coinvolgimento
CloudFront/ALB/Fargate	Trascurabile (1 API call)	~100ms
S3	Gestisce il trasferimento	Minuti (dipende dalla banda)
Aurora	Nessuno	N/A
Redis	Trascurabile (update stato)	~1ms
SQS	1 messaggio ricevuto	~1ms

1.4 FLUSSO 3 — Pipeline ETL (Elaborazione File Pesanti)

SQS ETL Queue → Fargate ETL Worker (poll) → S3 Raw (read file) → Transform in-memory → Aurora Writer (write)

Percorso dettagliato:

1. **Polling dalla coda SQS:** I Fargate ETL Workers eseguono un **long polling** sulla coda SQS (WaitTimeSeconds=20). Long polling riduce le chiamate API vuote e il costo, aspettando fino a 20 secondi prima di restituire una risposta vuota se non ci sono messaggi. Quando un messaggio è disponibile, un singolo worker lo riceve e il messaggio diventa **invisibile** agli altri worker per il periodo di VisibilityTimeout (configurato a un valore superiore al tempo massimo di elaborazione previsto, ad esempio 30 minuti per file da 2 GB).
2. **Download e parsing del file da S3:** Il worker scarica il file dal bucket Raw. Per file da 2 GB, il worker utilizza **streaming processing**: anziché caricare l'intero file in memoria, legge il file in chunks (stream) e processa riga per riga o blocco per blocco. Questo è possibile perché:
 - Per **CSV**: il parsing è naturalmente stream-friendly, riga per riga.
 - Per **Excel (XLSX)**: si utilizzano librerie di streaming parsing (come il SAX-based parsing) che non richiedono di caricare l'intero file in memoria.
 - Le Fargate Tasks ETL sono configurate con risorse adeguate (ad esempio 4 vCPU, 8 GB RAM) per gestire il processing, ma grazie allo streaming il consumo effettivo di memoria è una frazione della dimensione del file.
3. **Trasformazione e validazione:** Durante lo streaming, il worker applica:
 - Validazione dello schema (colonne attese, tipi di dato).
 - Pulizia dei dati (normalizzazione formati data, gestione valori nulli).
 - Trasformazioni business (calcoli derivati, aggregazioni).
 - I record invalidi vengono loggati e conteggiati, ma non bloccano l'intera elaborazione.
4. **Scrittura su Aurora Writer:** I dati trasformati vengono scritti sull'istanza **Aurora Writer** (unico endpoint di scrittura) utilizzando **batch inserts** (tipicamente 1000-5000 righe per batch) all'interno di transazioni. L'uso di batch insert anziché insert singoli riduce il numero di round-trip verso il database di ordini di grandezza e sfrutta le ottimizzazioni native di PostgreSQL per i bulk operations.
5. **Gestione degli errori e Dead Letter Queue (DLQ):**
 - Se il worker fallisce l'elaborazione (errore applicativo, timeout, crash del container), il messaggio torna visibile nella coda dopo il VisibilityTimeout e viene ritentato da un altro worker.

- Dopo un numero configurabile di tentativi falliti (tipicamente 3, impostato tramite `maxReceiveCount`), il messaggio viene automaticamente spostato nella **Dead Letter Queue (DLQ)**.
 - Un **CloudWatch Alarm** monitora la DLQ: se contiene messaggi (metrica `ApproximateNumberOfMessagesVisible` > 0), viene inviata una notifica via **SNS** al team operativo. Questo garantisce che nessun file "perso" passi inosservato.
6. **Aggiornamento stato e notifica:** Al completamento (successo o fallimento), il worker aggiorna lo stato del job in Redis (consultabile dalla dashboard) e, opzionalmente, può inviare un messaggio alla coda SQS Report per generare automaticamente un report sui dati appena importati.

Perché questo flusso evita colli di bottiglia sull'elaborazione:

- **Isolamento totale dai servizi utente:** i worker ETL sono un **servizio ECS separato** con il proprio auto-scaling, il proprio target group di risorse e la propria configurazione. Un picco di file da elaborare non sottrae risorse alla dashboard. Se vengono caricati 50 file da 2 GB contemporaneamente, la dashboard continua a rispondere normalmente.
- **Scale-to-zero e scale-up aggressivo:** il servizio ETL scala su una metrica personalizzata basata sulla **profondità della coda SQS** (`ApproximateNumberOfMessagesVisible`). Con zero messaggi in coda, non ci sono worker attivi (costo zero). Quando arrivano messaggi, il servizio scala fino a 20 task concorrenti, ciascuno capace di elaborare un file indipendentemente.
- **Fargate Spot per efficienza economica:** i worker ETL utilizzano **Fargate Spot**, che offre fino al 70% di sconto rispetto a Fargate standard. Poiché l'elaborazione è idempotente (se un worker Spot viene interrotto, il messaggio torna in coda e viene rielaborato da un altro worker), l'interruzione non causa perdita di dati, solo un leggero ritardo.
- **Scrittura solo su Aurora Writer, lettura solo su Aurora Reader:** la separazione dei path di lettura e scrittura garantisce che i batch insert pesanti dell'ETL non competano per le risorse con le query di lettura della dashboard.

1.5 FLUSSO 4 — Generazione Report Asincroni

Utente → Web/API (richiesta report) → SQS Report Queue → Fargate Report Worker

Report Worker → Aurora Reader (query dati) → Genera PDF → S3 Reports (salva)

Report Worker → SES (email con link al PDF)

Report Worker → Redis (aggiorna stato "completato")

Dashboard → Redis (poll stato) → mostra link download

Percorso dettagliato:

1. **Richiesta utente:** L'utente richiede la generazione di un report personalizzato dalla dashboard. La richiesta API raggiunge la Fargate Task Web/API, che:
 - Genera un reportJobId univoco.
 - Scrive lo stato iniziale "queued" in Redis.
 - Inserisce un messaggio nella coda SQS Report Jobs contenente i parametri del report (periodo, filtri, formato, email destinatario).
 - Restituisce immediatamente il reportJobId al frontend con HTTP 202 (Accepted).

L'utente non resta in attesa della generazione. La risposta è istantanea (~100ms) e la dashboard mostra uno stato "In elaborazione..." che si aggiorna consultando Redis.

2. **Elaborazione asincrona:** Un Fargate Report Worker preleva il messaggio dalla coda, interroga **Aurora Reader** per i dati necessari, genera il PDF in memoria, lo salva nel bucket **S3 Reports**, e invia un'email tramite **SES** contenente un link pre-signed temporaneo per il download del PDF.
3. **Aggiornamento stato:** Il worker aggiorna lo stato in Redis a "completed" con il link al PDF. La dashboard dell'utente, che effettua polling sullo stato tramite Redis, mostra il link di download.

Perché questo flusso evita colli di bottiglia:

La generazione di report complessi può richiedere secondi o minuti di elaborazione, con query pesanti sul database. Rendendola asincrona:

- La richiesta utente è sempre veloce (HTTP 202 in ~100ms).
- Le query pesanti vengono eseguite sulle **Aurora Reader replicas**, non sul Writer.
- I worker Report sono un servizio ECS separato con scale-to-zero, non consumano risorse quando non ci sono report da generare.
- Se vengono richiesti 100 report contemporaneamente, si accodano e vengono elaborati senza degradare la dashboard.

1.6 Flusso Trasversale — Networking e Sicurezza dei Dati in Transito

Tutti i flussi dati tra i servizi interni avvengono attraverso **VPC Endpoints (AWS PrivateLink)** per S3, SQS, ECR, CloudWatch e Secrets Manager. Questo significa che il traffico tra i container Fargate e questi servizi AWS **non esce mai dalla rete privata AWS**, non transita su Internet, e non attraversa i NAT Gateway (risparmiando costi di data transfer e riducendo la superficie d'attacco).

I **NAT Gateway** (uno per AZ per alta disponibilità) sono utilizzati esclusivamente per il traffico verso Internet che non può essere servito da VPC Endpoints (ad esempio, raggiungere API di terze parti o scaricare aggiornamenti).

SEZIONE 2: GIUSTIFICAZIONE DELLE SCELTE TECNOLOGICHE E ARCHITETTURALI

2.1 Principi Architettureali Guida

Prima di giustificare le singole scelte, è fondamentale esplicitare i **principi architetturali** che hanno guidato ogni decisione, derivati direttamente dai requisiti:

Principio	Derivato dal requisito	Implicazione pratica
Separazione dei piani di lavoro	"Responsiveness costante indipendentemente dal carico di background"	I workload interattivi (dashboard) e batch (ETL, report) devono essere fisicamente e logicamente isolati
Elasticità indipendente	"Indipendentemente dal carico di lavoro"	Ogni componente deve scalare autonomamente in base alla propria metrica di carico
Resilienza multi-AZ	"Garantire resilienza"	Nessun single point of failure; ogni componente ha un'istanza in almeno 2 Availability Zone
Fully managed services	"Cloud completamente vuoto" + operatività	Minimizzare il carico operativo scegliendo servizi gestiti dove possibile
Security by default	Best practice AWS Well-Architected	Encryption at rest e in transit ovunque; least privilege; audit trail completo

2.2 Scelte dell'Edge Layer

Route 53 come DNS

Scelta: Amazon Route 53 con Alias Record verso CloudFront.

Giustificazione: Route 53 è l'unica soluzione DNS che supporta **Alias Records** nativi verso risorse AWS (CloudFront, ALB, S3). Un Alias Record, a differenza di un CNAME, funziona anche sull'apex del dominio (acme.com, non solo sottodomini) e non aggiunge latenza di risoluzione aggiuntiva. Inoltre, le query DNS verso Alias Records per risorse AWS sono **gratuite**, mentre i CNAME generano costi. Route 53 offre anche health checks integrati e routing policies (failover, geolocation, weighted) che potrebbero servire in futuro per scenari multi-region.

CloudFront + WAF + ACM

Scelta: CloudFront come CDN con AWS WAF integrato e certificato TLS gestito da ACM.

Giustificazione composita:

- **CloudFront:** con oltre 450 Points of Presence globali, riduce la latenza per utenti geograficamente distribuiti. Per la dashboard, il beneficio principale non è solo il caching degli asset statici (che pure riduce il carico sull'ALB del 60-80% tipicamente) ma la **terminazione TLS all'edge**: il TLS handshake (che richiede 2-3 round trip) avviene nel PoP più vicino all'utente anziché nella Region AWS dove risiede l'ALB, risparmiando potenzialmente centinaia di millisecondi per ogni nuova connessione.
- **AWS WAF su CloudFront:** il WAF è posizionato su CloudFront anziché sull'ALB perché filtra il traffico malevolo **prima che entri nella VPC**. Attacchi DDoS di livello 7, bot, SQL injection e XSS vengono bloccati all'edge, proteggendo le risorse di backend senza che esse debbano nemmeno processare le richieste malevole. Il WAF su CloudFront beneficia anche di **AWS Managed Rules** aggiornate automaticamente contro le minacce emergenti.
- **ACM (AWS Certificate Manager):** fornisce certificati TLS gratuiti, con rinnovo automatico ogni 90 giorni senza intervento manuale. Elimina completamente il rischio operativo di certificati scaduti, che è una delle cause più comuni di downtime nei servizi web.

Alternative scartate:

- ALB esposto direttamente senza CloudFront: funzionerebbe, ma perderebbe la protezione edge, il caching CDN e la terminazione TLS distribuita. Inoltre, l'ALB sarebbe direttamente esposto ad attacchi.
- Cloudflare CDN: soluzione valida, ma introdurrebbe un vendor esterno, complicando la gestione IaC e la risoluzione dei problemi cross-vendor.

2.3 Scelte del Compute Layer

ECS Fargate (anziché EKS, EC2, Lambda)

Scelta: Amazon ECS su AWS Fargate come piattaforma di compute per tutti e tre i servizi (Web/API, ETL Workers, Report Workers).

Giustificazione dettagliata:

Perché container (e non Lambda):

- I worker ETL devono elaborare file da 2+ GB con processing che può durare 10-30 minuti. Lambda ha un **limite di 15 minuti di esecuzione**, 10 GB di memoria massima, e 512 MB di storage effimero (estendibile a 10 GB, ma comunque limitante per file da 2 GB). Lambda sarebbe stata inadatta per il workload ETL core.

- I container permettono di utilizzare qualsiasi runtime, libreria e strumento (ad esempio, librerie native per il parsing Excel), senza i vincoli del runtime Lambda.
- Per il servizio Web/API, i container mantengono connessioni persistenti con Aurora e Redis (connection pooling), mentre Lambda dovrebbe stabilire nuove connessioni ad ogni invocazione (cold start), creando potenziali problemi di esaurimento connessioni con il database.

Perché Fargate (e non EC2):

- **Zero gestione dell'infrastruttura:** con Fargate non è necessario gestire istanze EC2 (patching OS, sizing, AMI updates). In un cloud "completamente vuoto" come da requisiti, questo riduce significativamente la complessità operativa iniziale e ongoing.
- **Scaling granulare al singolo task:** Fargate scala aggiungendo o rimuovendo singoli container, non intere istanze EC2. Questo significa scaling più preciso e veloce (un nuovo task Fargate è pronto in ~30-60 secondi vs. 2-5 minuti per un'istanza EC2).
- **Scale-to-zero per i worker:** i servizi ETL e Report possono scalare a zero task quando non c'è lavoro, azzerando il costo. Con EC2, anche l'istanza più piccola in idle ha un costo continuo.
- **Isolamento a livello di microVM:** ogni Fargate task gira in una propria microVM (tecnologia Firecracker), offrendo un isolamento di sicurezza superiore ai container su EC2 condiviso.

Perché ECS (e non EKS):

- EKS (Kubernetes gestito) è una piattaforma più potente ma significativamente più complessa. Richiede la gestione del control plane Kubernetes, ha un costo fisso di ~\$73/mese per cluster, e la curva di apprendimento è notevolmente più ripida.
- Per tre servizi con pattern relativamente semplici (web server, queue consumer, queue consumer), ECS fornisce tutte le funzionalità necessarie (service discovery, auto-scaling, rolling deployments, integrazione nativa con ALB/SQS/CloudWatch) con una complessità operativa molto inferiore.
- La regola pratica: **EKS è giustificato quando si gestiscono >10-15 microservizi o si necessita di funzionalità specifiche di Kubernetes** (service mesh avanzata, CRD, multi-cluster federation). Per 3 servizi, ECS è la scelta più pragmatica.

Tre Servizi ECS Separati (e non un monolite containerizzato)

Scelta: Tre servizi ECS distinti — Web/API, ETL Workers, Report Workers.

Questa è la decisione architetturale più importante dell'intero design. Separando i workload in tre servizi indipendenti:

1. **Scaling indipendente basato su metriche diverse:**

- Web/API scala su **CPU al 60% e conteggio richieste ALB** → metriche di traffico utente.
- ETL Workers scala sulla **profondità della coda SQS ETL** → metriche di volume file in attesa.
- Report Workers scala sulla **profondità della coda SQS Report** → metriche di report richiesti.

Se vengono caricati 100 file contemporaneamente, solo i worker ETL scalano (fino a 20 task). I container Web/API non subiscono alcun impatto e mantengono la loro capacità dedicata al servizio degli utenti.

2. **Profili di risorse diversi:** i worker ETL possono essere configurati con 4 vCPU e 8 GB RAM (per il processing pesante), mentre i container Web/API possono usare 1 vCPU e 2 GB RAM (sufficienti per servire API). Senza separazione, tutti i container avrebbero dovuto avere le risorse del caso peggiore (ETL), sprecando risorse e denaro per le API.
3. **Deploy indipendenti:** un bug fix nel codice della dashboard può essere deployato senza toccare i worker ETL, e viceversa. Questo riduce il rischio e la frequenza di deploy di ogni singolo servizio.
4. **Fault isolation:** se un bug nell'ETL causa un crash loop, solo i worker ETL sono impattati. La dashboard e i report continuano a funzionare normalmente.

Fargate Spot per ETL Workers

Scelta: I worker ETL utilizzano Fargate Spot capacity.

Giustificazione: Fargate Spot offre uno sconto fino al 70% rispetto al prezzo standard. I worker ETL sono candidati ideali per Spot perché:

- Il workload è **interrompibile**: se un task Spot viene terminato (con un preavviso di 120 secondi), il messaggio SQS torna visibile nella coda e viene rielaborato da un altro worker. Non si perde lavoro, si ritarda solo.
- Il workload è **idempotente**: rielaborare lo stesso file produce lo stesso risultato (le scritture su Aurora sono all'interno di transazioni con upsert/conflict resolution).
- Il workload non è user-facing: un ritardo di qualche minuto nell'elaborazione ETL è accettabile.

I servizi Web/API e Report **non** usano Spot perché il web server deve essere sempre disponibile e i report hanno un'aspettativa di completamento ragionevole.

Auto-Scaling Configurations

Scelta:

- Web/API: Min 2, Max 10, trigger CPU 60% + ALB Request Count.
- ETL Workers: Min 0, Max 20, trigger SQS Queue Depth.
- Report Workers: Min 0, Max 10, trigger SQS Queue Depth.

Giustificazione:

- **Web/API Min 2:** il minimo di 2 task (uno per AZ) garantisce alta disponibilità anche con il carico minimo. Se un task o un'intera AZ ha un problema, l'altro task nell'altra AZ continua a servire. Il target CPU al 60% (non 80%) lascia headroom per assorbire spike improvvisi mentre lo scaling aggiunge nuovi task (che richiedono ~60 secondi per essere pronti).
- **ETL/Report Min 0 (Scale-to-Zero):** quando non ci sono messaggi nella coda, non ci sono worker attivi. Questo è critico per l'efficienza economica: se i file vengono caricati solo durante le ore lavorative, i worker sono attivi solo in quel periodo. La coda SQS funge da buffer: i messaggi si accumulano e i worker scalano in risposta. Il Max 20 per ETL è dimensionato per gestire picchi di caricamento senza ritardi eccessivi.

2.4 Scelte del Data Layer

Aurora PostgreSQL (anziché RDS PostgreSQL, DynamoDB, Redshift)

Scelta: Amazon Aurora PostgreSQL Serverless v2... in realtà Aurora Provisioned con cluster di Writer + Reader auto-scaling.

Giustificazione:

Perché PostgreSQL: i dati della dashboard sono **relazionali** (tabelle con relazioni, join, aggregazioni, vincoli di integrità). PostgreSQL è il database relazionale open-source più avanzato, con supporto nativo per JSON (query su dati semi-strutturati provenienti dai file), window functions (essenziali per analytics della dashboard), e un query planner sofisticato.

Perché Aurora (e non RDS PostgreSQL standard):

- **Performance:** Aurora è fino a 3x più veloce di PostgreSQL standard su AWS, grazie all'architettura di storage distribuito. Lo storage Aurora è separato dal compute, distribuito su 3 AZ con 6 copie dei dati, e offre repair automatico dei dati corrotti.
- **Failover più veloce:** in caso di failure del Writer, Aurora promuove un Reader a Writer in ~30 secondi (vs. 60-120 secondi di un RDS Multi-AZ failover).
- **Reader Auto-scaling:** le repliche di lettura possono scalare automaticamente da 1 a 5 (nella nostra configurazione) basandosi sul carico CPU o sul numero di connessioni. Questo è

fondamentale per il requisito di responsiveness costante: se la dashboard ha un picco di utenti, le repliche di lettura scalano per assorbire il carico aggiuntivo.

- **Storage auto-scaling:** lo storage Aurora cresce automaticamente in incrementi di 10 GB fino a 128 TB, senza necessità di pre-provisioning o downtime.

Separazione Writer/Reader

Scelta: tutte le scritture ETL vanno sul Writer endpoint; tutte le letture (dashboard, report) vanno sul Reader endpoint.

Giustificazione: questa è una forma di **CQRS (Command Query Responsibility Segregation)** a livello infrastrutturale. La motivazione è direttamente legata al requisito di responsiveness:

- Le operazioni ETL eseguono **batch insert pesanti** che generano I/O elevato, lock su tabelle e utilizzo CPU significativo sull'istanza database. Se le query della dashboard dovessero competere per le stesse risorse, i tempi di risposta della dashboard degraderebbero proporzionalmente al carico ETL.
- Separando i path, le query della dashboard vengono servite da repliche di lettura dedicate, il cui carico dipende solo dal numero di utenti della dashboard, non dal volume di file in elaborazione.
- La replica Aurora è **asincrona ma con lag tipicamente <100ms**, accettabile per una dashboard (i dati dell'ultimo file caricato appariranno con al massimo qualche secondo di ritardo).

ElastiCache Redis

Scelta: ElastiCache Redis con Multi-AZ failover per cache, sessioni e stato dei job.

Giustificazione tri-funzione:

1. **Cache delle query dashboard:** le query aggregate della dashboard (totali, grafici, tabelle) sono spesso identiche per molti utenti e cambiano solo quando arrivano nuovi dati. Redis permette di cachare i risultati con TTL di 30-60 secondi, riducendo il carico sulle Aurora Reader del 70-90% per le query più frequenti.
2. **Sessioni utente stateless:** memorizzare le sessioni in Redis anziché nei container rende ogni container Web/API identico e interscambiabile. Questo abilita: scaling orizzontale senza sticky sessions (che creano distribuzione disomogenea del carico), rolling deployments senza logout degli utenti, e recovery da failure di un container senza perdita di sessione.
3. **Stato dei job (ETL e Report):** Redis serve come store veloce per lo stato dei job asincroni. La dashboard può fare polling su Redis (O(1), sub-millisecondo) per mostrare lo stato di un upload o di un report senza interrogare Aurora. Questo pattern è chiamato "**job status tracking**" ed è fondamentale per l'UX dei workflow asincroni.

2.5 Scelte del Messaging Layer

SQS (anziché Kinesis, EventBridge, SNS per il decoupling)

Scelta: Amazon SQS Standard Queues con Dead Letter Queues per entrambe le pipeline (ETL e Report).

Giustificazione:

- **Decoupling produttore-consumatore:** SQS permette al produttore (S3 event o Web/API) e al consumatore (worker Fargate) di operare a velocità completamente diverse. Se arrivano 100 file in un minuto ma i worker possono elaborarne solo 5 al minuto, i 95 rimanenti restano in coda senza perdita. Senza SQS, il produttore dovrebbe aspettare il consumatore (coupling temporale) o i messaggi andrebbero persi.
- **Backpressure naturale e scale-to-zero:** la profondità della coda è la metrica di scaling per i worker. Con coda vuota → 0 worker (nessun costo). Con coda che cresce → più worker. Questo è il pattern "**queue-based load leveling**" del AWS Well-Architected Framework.
- **Dead Letter Queue:** i messaggi che falliscono ripetutamente vengono isolati nella DLQ anziché bloccare la coda principale. Questo previene lo scenario in cui un singolo file corrotto blocca l'intera pipeline ETL perché continua a fallire e rientrare in coda.

Perché non Kinesis: Kinesis Data Streams è ottimizzato per **streaming di dati in tempo reale ad alto throughput** (log streaming, clickstream, IoT). Il nostro caso è un pattern di **job queue** (messaggi discreti, ciascuno rappresenta un file da elaborare) dove l'ordine di elaborazione non è critico e la semantica "at-least-once con consumer che eliminano il messaggio dopo il processing" di SQS è il fit naturale. Kinesis aggiungerebbe complessità (gestione shard, checkpointing) senza benefici.

Perché non EventBridge: EventBridge è un event bus per il routing di eventi tra servizi AWS e applicazioni. Sarebbe appropriato se avessimo bisogno di fan-out complesso (lo stesso evento che trigga multiple azioni diverse) o di content-based routing. Per un semplice pattern producer→queue→consumer, SQS è più semplice e diretto.

2.6 Scelte di Networking e Sicurezza

VPC Design con Public/Private Subnet Multi-AZ

Scelta: VPC con CIDR /16, subnet pubbliche (ALB, NAT Gateway) e private (applicazione, database) distribuite su 2 AZ.

Giustificazione:

- **Principio del minimo privilegio di rete:** solo l'ALB e i NAT Gateway sono nelle subnet pubbliche (con route verso Internet Gateway). Tutti i container applicativi e i database sono in subnet private, irraggiungibili direttamente da Internet. Anche se un attaccante compromettesse il WAF e l'ALB, non potrebbe raggiungere direttamente i container o il database.

- **Multi-AZ per resilienza:** ogni componente è distribuito su almeno 2 Availability Zone (che sono data center fisicamente separati). Se un'intera AZ ha un'interruzione (evento raro ma documentato nella storia AWS), il servizio continua a operare dall'altra AZ. Questo soddisfa direttamente il requisito di resilienza.
- **NAT Gateway per AZ:** un NAT Gateway per AZ evita che il traffico in uscita attraversi AZ boundaries (che è sia un costo aggiuntivo sia un potenziale punto di failure — se l'AZ del NAT Gateway va down, le subnet private nell'altra AZ perdono l'accesso a Internet).

VPC Endpoints (AWS PrivateLink)

Scelta: VPC Endpoints per S3, SQS, ECR, CloudWatch e Secrets Manager.

Giustificazione triplice:

1. **Sicurezza:** il traffico verso questi servizi AWS non esce mai dalla rete privata AWS. Senza VPC Endpoints, una chiamata API a S3 da una Fargate Task nella subnet privata dovrebbe uscire attraverso il NAT Gateway, traversare Internet, ed entrare nel servizio S3 pubblico. Con i VPC Endpoints, il traffico resta completamente all'interno del backbone AWS.
2. **Costo:** il NAT Gateway costa \$0.045/ora + \$0.045/GB di dati processati. I worker ETL che scaricano file da 2 GB da S3 attraverso il NAT Gateway genererebbero costi di data transfer significativi. I VPC Endpoints per S3 (Gateway Endpoint) sono **gratuiti**. Considerando che l'ETL potrebbe processare centinaia di GB al giorno, il risparmio è sostanziale.
3. **Performance:** il path di rete è più diretto e stabile, con latenza inferiore e throughput superiore rispetto al percorso via NAT Gateway.

Secrets Manager + KMS

Scelta: AWS Secrets Manager per credenziali database e API keys, con rotazione automatica. KMS per le chiavi di crittografia.

Giustificazione:

- **Secrets Manager:** elimina la necessità di hardcodare credenziali nel codice, nelle variabili d'ambiente dei task definition, o in file di configurazione. I container Fargate recuperano i secret a runtime attraverso il VPC Endpoint, con il task role IAM che autorizza l'accesso solo ai secret specifici necessari. La **rotazione automatica** dei secret (ogni 30-90 giorni) garantisce che le credenziali database vengano aggiornate senza intervento manuale e senza downtime (Secrets Manager coordina l'aggiornamento della password Aurora con l'aggiornamento del secret).
- **KMS:** una singola customer-managed key (CMK) o chiavi separate per dominio crittografano: i dati at rest nei bucket S3 (SSE-KMS), i volumi Aurora (encryption at rest), i secret in Secrets Manager, e i messaggi in SQS (encryption at rest). La centralizzazione delle chiavi in KMS permette audit completo (via CloudTrail) di chi ha accesso a cosa e quando.

2.7 Scelte di Observability

CloudWatch + X-Ray + CloudTrail + VPC Flow Logs

Scelta: stack di observability interamente AWS-native.

Giustificazione strutturata per pillar:

- **Metriche e Allarmi (CloudWatch):** raccoglie metriche da tutti i servizi AWS utilizzati (CPU/memoria Fargate, latenza ALB, profondità code SQS, connessioni Aurora, hit rate Redis) in un unico punto. Gli allarmi sono configurati per notificare via SNS quando: la DLQ contiene messaggi (file falliti), il CPU del Web/API supera l'80% per >5 minuti (scaling potenzialmente insufficiente), l'età del messaggio più vecchio in coda supera una soglia (ritardo nell'elaborazione), il numero di errori 5xx dell'ALB supera una soglia.
- **Distributed Tracing (X-Ray):** per le richieste che attraversano Web/API → SQS → Worker → Aurora, X-Ray fornisce una vista end-to-end della latenza di ogni componente. Questo è essenziale per diagnosticare problemi di performance: se la dashboard è lenta, X-Ray mostra immediatamente se il collo di bottiglia è nel container, nella query Aurora, nella chiamata Redis o nel network.
- **Audit Trail (CloudTrail):** registra ogni chiamata API AWS effettuata nell'account. È fondamentale per security forensics (chi ha accesso a quale bucket? chi ha modificato una security group?) e compliance. CloudTrail è un requisito per molti framework di compliance (SOC 2, ISO 27001, GDPR).
- **VPC Flow Logs:** registrano il traffico di rete accettato e rifiutato nelle interfacce di rete della VPC. Utili per diagnosticare problemi di connettività e per rilevare pattern di traffico anomali.

Alternative scartate: Datadog, New Relic, Grafana Cloud — sono soluzioni eccellenti ma introducono un vendor esterno, costi aggiuntivi significativi, e la necessità di esfiltrare dati dall'account AWS. Per una piattaforma greenfield, lo stack nativo AWS è sufficiente e minimizza la complessità. Si può evolvere verso soluzioni terze quando la complessità operativa lo giustifica.

SEZIONE 3: INFRASTRUTTURA AS CODE — RIFERIMENTO TERRAFORM

3.1 Struttura dei Moduli

L'infrastruttura è interamente gestita tramite Terraform ($\geq 1.7.0$) con AWS Provider $\sim > 5.60$. Il codice è organizzato in otto moduli indipendenti che rispecchiano i layer architetturali: networking, security, storage, messaging, database, compute, edge e observability. Ogni modulo espone variabili di input e output tipizzati per garantire il corretto passaggio di riferimenti tra moduli (ARN, endpoint, security group ID).

3.2 Gestione degli Ambienti

Sono supportati tre ambienti (dev, staging, prod), ciascuno con la propria directory in environments/ e il relativo file terraform.tfvars. Il remote state è conservato su S3 con locking DynamoDB. La separazione degli state file per ambiente garantisce che un'operazione su dev non possa impattare la produzione.

3.3 Provider Multi-Region

è configurato un provider AWS principale nella region di deployment (default eu-south-1, Milano) e un provider alias obbligatorio in us-east-1, necessario per le risorse globali CloudFront: il certificato ACM per CloudFront e il WAF con scope CLOUDFRONT devono essere creati esclusivamente in us-east-1, indipendentemente dalla region applicativa scelta. Entrambi i provider applicano default_tags automatici su tutte le risorse per garantire la tracciabilità dei costi.

3.4 Variabili e Validazioni

Tutte le variabili sensibili (environment, aws_region) sono dotate di validation block Terraform che impediscono il deploy con valori non consentiti. Le variabili per le immagini Docker (*_image) accettano URI ECR completi con tag espliciti; l'uso del tag :latest è sconsigliato in produzione per garantire la riproducibilità dei deploy. I capacity limits (min/max) per i tre servizi ECS sono esternalizzati come variabili per permettere il tuning senza modifiche al codice Terraform.

3.5 Tagging Automatico e Cost Allocation

Ogni risorsa AWS creata da Terraform riceve automaticamente i seguenti tag tramite default_tags del provider: Project (nome progetto), Environment (dev/staging/prod), ManagedBy (terraform), Owner (platform-team), Region (region di deployment). Questa convenzione abilita la cost allocation per ambiente e progetto tramite AWS Cost Explorer e semplifica la ricerca delle risorse nella console AWS.